RELATÓRIO TÉCNICO

LIA — Agente SDR Imobiliário com Inteligência Artificial

Hackathon • Pós-graduação IA para Desenvolvedores • FIAP

Prova de conceito de atendimento comercial imobiliário com agente conversacional, qualificação determinística de leads, busca híbrida de imóveis, agendamento, dashboard e follow-up automático no Chat WEB.

Repositório: https://github.com/AlmeidaLima/fiap-sdr-imobiliario

Vídeo de demonstração: disponível na entrega do projeto; inserir URL pública quando publicada.

Natureza: projeto acadêmico / prova de conceito (POC).

1. Contexto e objetivo

O projeto LIA foi desenvolvido para o Hackathon da pós-graduação FIAP — IA para Desenvolvedores. Seu objetivo é apoiar a etapa inicial do relacionamento entre potenciais compradores ou locatários e profissionais do mercado imobiliário. A solução organiza as informações de uma conversa, identifica preferências e intenção comercial, recomenda imóveis cadastrados e prepara a continuidade do atendimento pelo corretor.

A LIA é uma POC acadêmica: demonstra integração entre inteligência artificial e regras de negócio auditáveis, mas não representa um CRM de produção nem substitui a atuação comercial humana.

1.1 Capacidades implementadas

- Chat web com histórico persistido e manutenção de contexto;
- identificação de intenção comercial, preferências e urgência;
- qualificação progressiva e Lead Score determinístico com classificação FRIO, MORNO ou QUENTE;
- busca estruturada, semântica e híbrida sobre imóveis persistidos;
- seleção de imóvel, agendamento de visita/reunião e handoff para o corretor;
- dashboard de consulta dos leads e follow-up automático por mensagem no Chat WEB.

2. Descrição do fluxo de atendimento

2.1 Início e qualificação da conversa

O cliente conversa com a LIA pela interface web. O backend registra os turnos, identifica a intenção comercial (compra, aluguel ou investimento, quando detectável) e extrai informações como região, faixa de preço, quartos, banheiros e urgência. O estado persistido permite continuar a conversa sem repetir dados já informados.

2.2 Pontuação comercial

As informações extraídas alimentam serviços determinísticos de qualificação e pontuação. O Lead Score varia de 0 a 100 e sustenta a classificação FRIO, MORNO ou QUENTE. A decisão de pontuação não é delegada ao LLM: regras de negócio permitem previsibilidade, rastreabilidade e testes automatizados.

2.3 Recuperação e seleção de imóveis

A consulta combina restrições estruturadas (por exemplo, localização, preço e número de quartos) com recuperação vetorial por embeddings. Os resultados correspondem a registros do banco de dados e são exibidos em cards. O cliente pode selecionar uma opção e seguir para a marcação de visita ou reunião.

2.4 Agendamento e encaminhamento

Após a seleção, o sistema registra o agendamento e fornece um handoff estruturado ao corretor, reunindo dados do lead, preferências e contexto da conversa. O dashboard disponibiliza uma visão somente leitura da oportunidade.

2.5 Follow-up automático

O agendamento do follow-up é realizado por API, inclusive via Swagger. Um worker separado verifica periodicamente os contatos pendentes e, no horário programado, persiste uma mensagem do assistente na conversa do lead. O Chat consulta novas mensagens e exibe o contato automaticamente. A implementação atual entrega mensagens somente no canal WEB; não há integração com WhatsApp ou e-mail.

3. Arquitetura da solução

A arquitetura separa interface, API, orquestração conversacional, serviços de domínio, repositórios e persistência. O backend FastAPI expõe endpoints HTTP; LangGraph coordena as etapas da conversa; PostgreSQL com pgvector armazena dados transacionais e vetores. O worker de follow-up executa independentemente do fluxo de requisições web.

Camada	Componente	Responsabilidade
Interface	Next.js / React / TypeScript	Chat e dashboard dos leads
API e domínio	Python / FastAPI / services	Endpoints, validações e regras de negócio
Orquestração de IA	LangGraph	Fluxo conversacional e decisões de encaminhamento
Geração de texto	LLM local opcional	Respostas em linguagem natural, com fallback determinístico
Busca e dados	PostgreSQL / pgvector	Imóveis, leads, histórico, embeddings e consultas
Processamento assíncrono	Follow-up worker	Entrega programada de mensagens no Chat WEB

O projeto utiliza Docker Compose para execução dos serviços e Alembic para evolução controlada do esquema do banco. A documentação de arquitetura está em docs/ARCHITECTURE.md.

4. Modelos e técnicas de inteligência artificial

Técnica / modelo	Aplicação no projeto
LangGraph	Orquestra o estado e as etapas da conversa com o lead.
LLM local opcional (servidor compatível com API OpenAI)	Produz respostas em linguagem natural quando configurado e disponível.
Fallback determinístico	Mantém o atendimento funcional quando o LLM não está disponível.
Embeddings (intfloat/multilingual-e5-small)	Representação vetorial dos imóveis para recuperação semântica.
pgvector e busca híbrida	Combinam filtros de negócio e correspondência por significado.
Lead Score determinístico	Classifica oportunidades sem depender de inferência do modelo de linguagem.

4.1 RAG aplicado ao catálogo de imóveis

O RAG implementado recupera descrições vetorizadas dos imóveis cadastrados. Não se trata de uma base documental genérica de perguntas frequentes ou arquivos fragmentados: a recuperação é orientada ao catálogo persistido e pode ser combinada a filtros estruturados. O dataset acadêmico de referência é Curitiba Apartment Real Estate Data, com origem atribuída ao conjunto Curitiba Apartment Prices (Kaggle, wittmannf).

4.2 Limites do uso de LLM

O servidor de linguagem é externo ao Docker Compose e opcional. Na sua ausência, ou em caso de timeout, o sistema usa respostas determinísticas. A qualificação comercial, o cálculo de score, as consultas, o agendamento e o follow-up não dependem da disponibilidade do modelo generativo.

5. Persistência, APIs e processamento

A aplicação mantém histórico conversacional e entidades de negócio em PostgreSQL. Migrations Alembic estabelecem o esquema. A importação do dataset e a indexação dos embeddings possuem scripts próprios, permitindo reproduzir a preparação da base.

Recurso / endpoint	Finalidade
POST /api/v1/conversations/turn	Processar mensagens do cliente.
GET /api/v1/properties/hybrid-search	Buscar imóveis combinando filtros e vetores.
POST /api/v1/appointments	Registrar visita ou reunião.
GET /api/v1/handoffs/{conversation_id}	Consultar o encaminhamento ao corretor.
GET /api/v1/dashboard/leads	Consultar leads no dashboard.

Recurso / endpoint	Finalidade
POST /api/v1/follow-ups	Programar novo contato com o lead.
GET /api/v1/follow-ups/{follow_up_id}	Verificar situação do follow-up.

O worker verifica os follow-ups vencidos a cada 30 segundos. Na entrega bem-sucedida, mensagem, tentativa de entrega e atualização de status são persistidas atomicamente. O Chat consulta novas mensagens aproximadamente a cada 3 segundos. O processamento evita duplicações por controle transacional e de status.

6. Resultados e evidências da POC

O fluxo implementado permite demonstrar um ciclo comercial completo: primeiro contato, identificação das preferências, pontuação, busca de imóveis reais, seleção, agendamento, consulta pelo corretor e mensagem de retomada enviada automaticamente ao Chat. O comportamento é observável tanto pela interface quanto pelos endpoints e registros persistidos.

6.1 Saídas observáveis

- Histórico de mensagens e estado da conversa;
- Lead Score e classificação comercial no painel de qualificação;
- cards com imóveis cadastrados e seleção de uma opção;
- registro de visita ou reunião e handoff para o corretor;
- dashboard com lista e detalhamento de leads;
- follow-up concluído, com mensagem exibida automaticamente no Chat e status consultável pela API.

6.2 Testes e verificações

O repositório contém testes automatizados de backend (Pytest) e frontend (Vitest), além de comandos para lint e build. A validação contempla regras comerciais determinísticas, APIs, fluxos de conversa e componentes da interface. Este relatório não declara métricas quantitativas finais de cobertura ou taxa de aprovação sem anexar o resultado de uma execução identificável.

7. Execução e reprodução

Pré-requisitos: Git, Docker Desktop com Docker Compose v2 e acesso à rede para eventual download do modelo de embeddings. O LLM local é opcional. Na raiz do repositório, copie .env.example para .env e configure os valores de desenvolvimento.

1. docker compose up -d postgres
2. docker compose run --rm backend alembic upgrade head
3. docker compose run --rm backend python -m scripts.import_curitiba_properties
4. docker compose run --rm backend python -m scripts.index_property_embeddings --batch-size 100
5. docker compose up --build -d

Aguarde o PostgreSQL aceitar conexões antes de aplicar as migrations. Com a configuração padrão, o Chat fica em http://localhost:3000, o dashboard em /dashboard e a documentação Swagger em http://localhost:8000/docs. Para instruções completas e comandos de testes, consulte o README do repositório.

8. Limitações e pontos de atenção

- A solução é uma prova de conceito, sem autenticação/autorização ou recursos de alta disponibilidade próprios de produção.
- O dashboard é somente leitura e não oferece gestão completa de CRM.
- O follow-up automático é restrito ao Chat WEB e seu agendamento é feito pela API.
- O LLM local não é iniciado automaticamente pelo Compose e pode estar indisponível.
- A recuperação vetorial utiliza o catálogo de imóveis, não uma base genérica de documentos e FAQs.
- Não há máquina comercial completa de estados do lead nem integração de mensagens por WhatsApp ou e-mail.

9. Evoluções futuras

Como próximos passos, são possíveis integrações reais de mensageria, autenticação e perfis de acesso, expansão da base de conhecimento RAG, políticas avançadas de retry e backoff para contatos, estados comerciais mais completos e avaliação formal de qualidade e desempenho em cenários de produção.

10. Conclusão

A LIA demonstra a aplicação prática de inteligência artificial em um fluxo comercial imobiliário, combinando orquestração conversacional, recuperação de imóveis por significado e regras de negócio determinísticas. A separação entre geração de linguagem e decisões comerciais críticas contribui para a previsibilidade e a capacidade de validação da POC. O projeto integra atendimento, qualificação, busca, agendamento, acompanhamento e follow-up em uma solução acadêmica reproduzível.

11. Referências e materiais de entrega

Código-fonte: https://github.com/AlmeidaLima/fiap-sdr-imobiliario

Documentação: README.md; docs/ARCHITECTURE.md; docs/TRACEABILITY.md; docs/REQUIREMENTS.md.

Vídeo: arquivo de demonstração da LIA, a ser disponibilizado no repositório ou em link público da entrega.

Equipe: preencher nomes completos e RMs dos integrantes antes da submissão.

Uso: material acadêmico desenvolvido para o Hackathon da pós-graduação FIAP — IA para Desenvolvedores.